

SC4064 Assignment 2 Report

Pu Fanyi
U2220175K
FPU001@e.ntu.edu.sg

All programs were compiled with `nvcc -O2 -std=c++20 -lcusparse` and tested on the platform in Table 1. Kernel timing uses CUDA events; each configuration is preceded by a context warm-up.

Component	Details
OS	Ubuntu 24.04.4 LTS (Noble Numbat)
Kernel	Linux 5.14.0-284.25.1.el9_2.x86_64
CPU	2× Intel Xeon Gold 6448Y (128 threads)
RAM	2.0 TiB
GPU	8× NVIDIA H100 80 GB HBM3
NVIDIA Driver	550.90.07
CUDA Toolkit	13.1 (V13.1.115)
GCC/G++	14.2.0

Table 1: Experimental environment.

1 Problem Setup and Implementation

We solve the 2D wave equation $\partial^2 u / \partial t^2 = c^2 \nabla^2 u$ on $[0, L]^2$ with $c = 1$, $\Delta x = \Delta y = 0.01$, $\Delta t = 0.005$, giving $\lambda^2 = c^2 \Delta t^2 / \Delta x^2 = 0.25$ ($\lambda = 0.5 \leq 1/\sqrt{2}$). Initial condition: $u(0, x, y) = \sin(\pi x) \sin(\pi y)$, $\partial_t u(0) = 0$; Dirichlet boundaries $u = 0$. The analytical solution $u(t, x, y) = \cos(c\sqrt{2}\pi t) \sin(\pi x) \sin(\pi y)$ is used for verification.

Implementations. Three methods share the same initialization and first-step kernel. **(A1)** Global memory: each thread computes one interior point directly from global memory. **(A2)** Shared memory: a 2D tile with 1-cell halo is loaded cooperatively into shared memory; border threads load the halo elements; a `__syncthreads()` precedes stencil computation. **(B)** cuSPARSE: the discrete Laplacian is stored as a CSR matrix (≤ 5 non-zeros per row); each time step calls `cusparseSpMV` followed by a fused vector-update kernel.

2 Block Size Study ($L = 1$)

All results use 200 time steps (199 timed kernel calls). The 101×101 grid ($99^2 = 9,801$ interior points) is very small for the H100.

Block	Global (ms)	Shared (ms)	BW _G (GB/s)	BW _S (GB/s)
(8, 8)	0.489	0.662	191.3	141.3
(16, 16)	0.479	0.639	195.6	146.6
(32, 8)	0.481	0.652	194.6	143.5
(32, 32)	0.500	0.647	187.1	144.7

Table 2: Total kernel time for 199 steps ($L = 1$, 101×101 grid). Max error: 8.81×10^{-5} for all.

Analysis. Differences are small because the grid is tiny. (16, 16) (256 threads) is marginally best: it provides enough warps (8) for latency hiding without the occupancy penalty of (32, 32) (1024 threads, only 2 resident blocks per SM). On this grid, global memory *outperforms* shared memory by $\sim 1.3\times$ because the entire working set ($3 \times 101^2 \times 8 = 245$ KB) fits in L2 cache, making the extra shared-memory bookkeeping (halo loads, synchronization) pure overhead.

3 Scaling Study

The domain is enlarged to $[0, L]^2$ for $L \in \{1, 2, 4, 8\}$ while keeping $\Delta x, \Delta t$ fixed. Block size (16, 16).

L	N_x	Interior points	Global (ms)	Shared (ms)	cuSPARSE (ms)	BW _G (GB/s)	BW _S (GB/s)
1	101	9 801	0.47	0.64	6.02	200	147
2	201	39 601	0.49	0.66	2.87	778	575
4	401	159 201	0.58	0.74	3.51	2 610	2 063
8	801	638 401	1.06	1.40	8.04	5 735	4 352

Table 3: Total kernel time for 199 steps (block 16×16). Max error: 8.81×10^{-5} for all.

Runtime scaling. From $L=1$ to $L=8$ the interior points increase $65\times$, while global-memory time grows only $2.3\times$. For small grids ($L \leq 2$), kernel launch overhead dominates; true scaling emerges at $L \geq 4$. From $L=4$ to $L=8$ (a $4\times$ workload increase), global-memory time increases $1.8\times$ — sub-linear because larger grids amortise fixed overhead.

Effective bandwidth. Using the assignment’s formula ($6 \times 8 = 48$ bytes per point), the global-memory kernel reaches **5 735 GB/s** at $L=8$, far exceeding the H100’s HBM3 peak ($\sim 3\,350$ GB/s). This is because the total working set ($3 \times 801^2 \times 8 \approx 15.4$ MB) fits entirely in the H100’s 50 MB L2 cache, so most accesses are served at L2 bandwidth. The kernel is *memory-bandwidth-bound*: the 5-point stencil performs only 7 FLOPs per 48 bytes (0.15 FLOP/byte), well below the H100’s compute-to-bandwidth ratio.

cuSPARSE comparison. The library-based solver is $6\text{--}13\times$ slower than the custom stencil. The SpMV must traverse CSR metadata (row pointers, column indices) and performs indirect memory accesses, which undermine cache locality. Additionally, each time step requires *two* kernel launches (SpMV + vector update) versus one for the stencil. For structured-grid problems where the stencil pattern is known at compile time, a hand-written kernel eliminates all indexing overhead.

4 Visualization

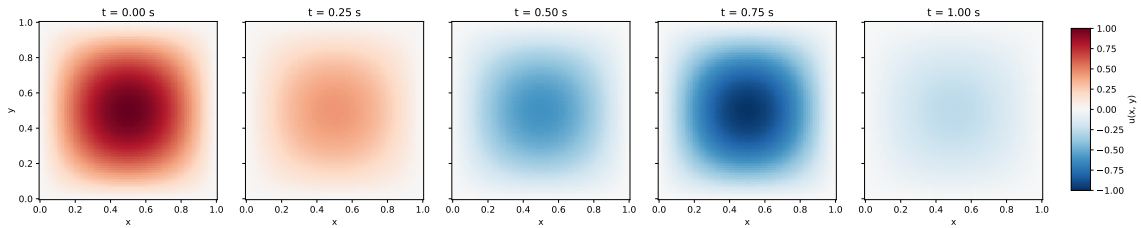


Figure 1: Wave field evolution on $[0, 1]^2$ at five time instants. The wave oscillates as $\cos(\sqrt{2}\pi t)$: positive at $t=0$, crossing zero near $t \approx 0.35$ s, and reaching maximum negative amplitude at $t \approx 0.71$ s.

5 Discussion

Correctness. The L_∞ error of 8.81×10^{-5} is consistent across all three methods and all domain sizes, confirming both algorithmic correctness and that the error depends only on Δx and Δt (which are fixed throughout the scaling study), as expected for a second-order finite-difference scheme.

Shared memory overhead. The shared-memory kernel is $\sim 1.3\times$ slower than the global-memory kernel across all domain sizes. Because these grids are small enough to fit in L2 cache, the five-point stencil already enjoys near-optimal data reuse without shared memory. The shared-memory version incurs extra cost for halo loading (4 conditional branches per thread) and a `__syncthreads()`



Figure 2: Surface plots at $t = 0$ (left, initial condition $\sin(\pi x) \sin(\pi y)$) and $t = 0.75$ s (right, amplitude inverted to ≈ -0.71).

barrier per step, which are not compensated by reduced global-memory traffic. On larger grids (e.g., 4096×4096) that exceed L2 capacity, the shared-memory tiling would show a clear advantage.

Block size. Differences are modest ($< 6\%$) because all tested configurations produce enough warps for adequate occupancy on these small grids. $(16, 16)$ balances thread count (256) and block count, allowing the scheduler to distribute work evenly across SMs.

Memory-bound vs. compute-bound. The 5-point stencil update performs 7 FLOPs (4 additions for neighbours, 1 multiply-add for the centre, 1 subtraction, 1 FMA for the final update) while transferring at least 48 bytes (5 reads of `u_curr`, 1 write of `u_next`; the read of `u_prev` adds another 8 bytes in practice). The operational intensity of ~ 0.15 FLOP/byte places the kernel firmly in the memory-bound regime of the H100 roofline (~ 51 TFLOPS FP64 peak, ~ 3.35 TB/s HBM3). Consequently, optimisation effort should focus on reducing memory traffic (e.g., temporal blocking) rather than increasing arithmetic throughput.

Linear scaling analysis. The table below shows per-step kernel time and the scaling ratio relative to $L = 1$. If scaling were perfectly linear with grid points, the ratio column would match the workload multiplier.

L	Workload $\times$	Global $\mu\text{s}/\text{step}$	Time $\times$	Efficiency
1	1 $\times$	2.35	1.0 $\times$	—
2	4 $\times$	2.44	1.04 $\times$	3.9 $\times$
4	16 $\times$	2.93	1.25 $\times$	12.8 $\times$
8	65 $\times$	5.34	2.27 $\times$	28.6 $\times$

Table 4: Per-step timing and scaling efficiency (global memory, block 16×16). “Efficiency” = workload multiplier / time multiplier, showing how many $\times$ more work is done per unit time relative to $L = 1$.

At $L = 8$ the kernel processes $65\times$ more interior points but takes only $2.27\times$ longer per step, yielding a $28.6\times$ throughput gain. This dramatic sub-linear scaling arises because the $L = 1$ grid (9801 points) drastically under-utilises the H100’s 132 SMs and memory subsystem; kernel launch overhead alone accounts for $\sim 2 \mu\text{s}$. As the grid grows, fixed costs are amortised and more SMs are saturated. The H100 architecture is designed for grids of millions of points; at the sizes tested here, even $L = 8$ leaves significant headroom.

cuSPARSE non-monotonic timing. Interestingly, cuSPARSE is *faster* at $L = 2$ (2.87 ms) than $L = 1$ (6.02 ms). This reflects cuSPARSE’s internal tuning: the library selects algorithms based on matrix size, and very small matrices ($\sim 10\text{K}$ rows) may trigger a less optimised code path. At $L \geq 4$ the cost grows proportionally with nnz as expected.